Current Production CMG Benchmark Qualification

Rust versus the official MATLAB/C MEX package

CMG maintainers

Updated 29 August 2026

Abstract

This report describes the current production comparison between the Rust package and the official MATLAB solver with its default C MEX build. Five deterministic connected graphs, each with one million vertices and between about one and eight million canonical edges, were evaluated at 1, 8, 16, and 32 application CPUs. Each reported point is the median of 7 measurements after 2 warmups. At 16 CPUs, geometric-mean Rust/MATLAB ratios are 0.182 for hierarchy setup, 0.379 for stationary preconditioner application, 0.388 for reused-preconditioner PCG, 0.391 for setup plus one solve, and 0.227 for process peak resident memory. Rust is faster in setup-plus-solve for every family at every tested CPU count. Sixteen CPUs minimize Rust's aggregate single-right-hand-side latency on the tested hardware; 32 CPUs remain supported but regress for every family.

1 Decision summary

Measured default on Intel Xeon Gold 6242: 16 application CPUs for one large right-hand side. At that setting, Rust takes 1.334 seconds in geometric mean versus 3.412 seconds for MATLAB, a Rust/MATLAB ratio of 0.391. Rust's one-to-16-CPU speedup is 1.87x, versus 1.23x for MATLAB. Moving from 16 to 32 CPUs multiplies Rust total time by 1.35 and MATLAB total time by 1.05. Grid and weak-community inputs are individually fastest in Rust at 8 CPUs; the other three families are fastest at 16.

The principal comparison is the complete native package workflow, not an isolated kernel. Rust's setup-plus-solve total includes hierarchy construction, parallel-plan construction when routed, and PCG. MATLAB's total includes its official `cmg_precondition` workflow followed by `pcg`. The matrix is already resident, so binary input and graph or sparse-matrix assembly are excluded. Lower Rust/MATLAB ratios favor Rust throughout this report.

2 Benchmark configuration and protocol

The benchmark is accepted run `20260828T021628Z-6fe9be77084a-b2v1-rust-matlab-current`, SGE array 7341600. It evaluates Rust source `6fe9be77084a60cca330760361dd4c7addc77ccf` against the official CMG source pinned at `19752fc102f8cae8e34f66457bfaccb1aaa60375`. The figures are generated from the run's 40 reduced configuration rows and checked numerically against the maintained compact record before the PDF is compiled.

Table 1: Controlled study design.

Item	Current qualification
Implementations	Current Rust production package; official MATLAB package with its default C MEX build
Toolchains	Rust 1.98.0; MATLAB 2026a
Hardware	Intel Xeon Gold 6242; whole-node 32-slot SGE allocation; application limits of 1, 8, 16, and 32 CPUs
Graphs	Weighted path; 2D grid; worker-firm degree 3; worker-firm degree 16; weak community; all one million vertices
Timing	2 warmups followed by 7 measured repetitions; medians reported; per-configuration bootstrap intervals shown for total time
Accuracy	Native stopping logic in each package; tolerance 1e-08; at most 1,000 iterations; independent residual and backward-error checks retained
Identity	Shared deterministic graph, right-hand side, truth vector, and metadata bytes; hashes recorded per task

2.1 Measured stages

Hierarchy/preconditioner setup
: CMG hierarchy and terminal factor construction. Rust reports reusable parallel-plan construction separately, and the plan is included in total time.

Stationary preconditioner apply
: One normalized CMG application with an existing hierarchy and workspace. Fast applications are looped and divided by the loop count.

Reused-preconditioner PCG
: One PCG solve after setup. The implementations retain their native stopping rules, so timings do not hold iteration counts or residual definitions fixed.

Setup + plan + solve
: Fresh hierarchy setup, Rust plan setup when applicable, and one solve. This is the main user-facing latency measure.

Peak RSS
: Process peak resident set size from `/usr/bin/time -v`; this includes input and runtime storage and is not an allocation-by-allocation decomposition.

3 Rust and MATLAB on the same timing axes

Figure 1 puts the Rust and MATLAB lines together for every primary stage. It is the quickest absolute comparison: Rust setup is substantially lower at every CPU count; PCG is close to MATLAB at one CPU and separates at 8 and 16 CPUs; both implementations show limited end-to-end scaling because hierarchy work and overhead remain important.

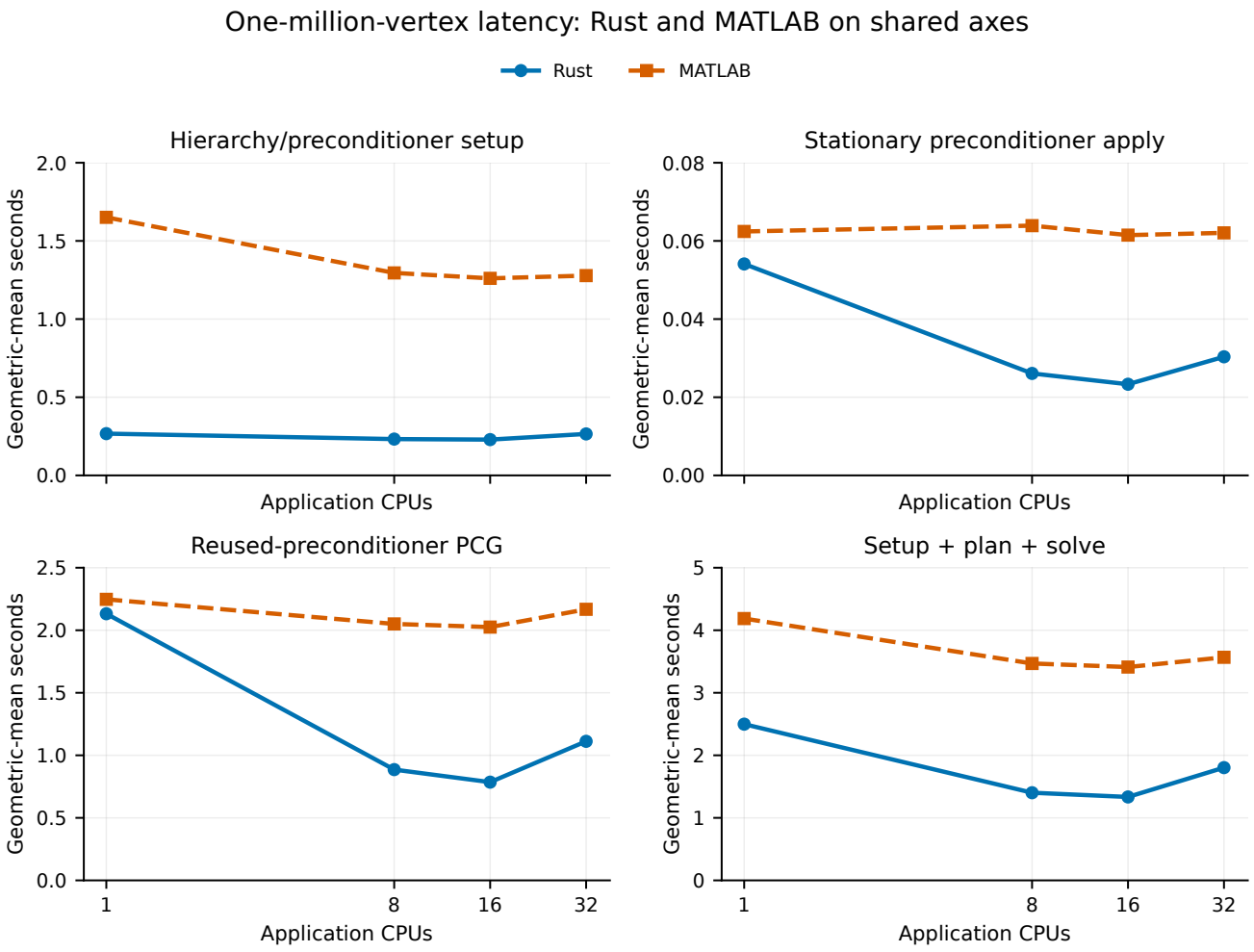
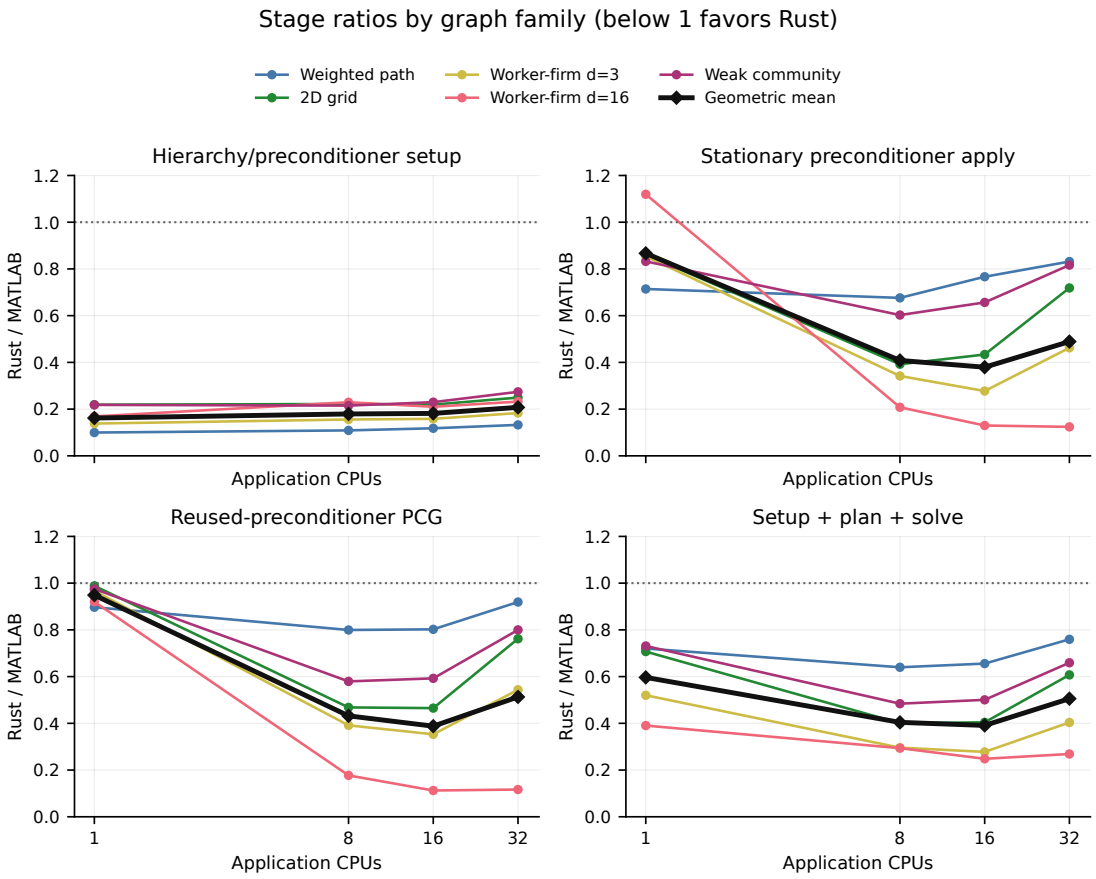


Figure 1: Geometric-mean stage times across the five graph families. Rust is the solid blue line with circles; MATLAB is the dashed orange line with squares. Both implementations share a zero-based linear time axis within each panel.

4 Stage ratios

Figure 2 shows the corresponding Rust/MATLAB ratios for each family and the geometric mean. The ratio view makes heterogeneity visible without hiding the absolute scale. At 16 CPUs, the geometric-mean ratios are 0.182, 0.379, 0.388, and 0.391 for setup, apply, PCG, and total time. One one-CPU stationary-apply point is slightly above one; the main total is below one for all 20 family/CPU combinations.



Aggregate geometric means (Rust / MATLAB)

CPUs	Setup	Apply	PCG	Total	Peak RSS
1	0.162	0.867	0.949	0.597	0.138
8	0.179	0.408	0.432	0.404	0.233
16	0.182	0.379	0.388	0.391	0.227
32	0.208	0.489	0.513	0.505	0.223

Figure 2: Family-level Rust/MATLAB timing ratios plus the geometric mean. The dotted reference is parity; values below one favor Rust. The table gives the exact geometric-mean ratios at each CPU count.

5 Family-level setup-plus-solve comparisons

The complete total-time curves in Figure 3 place MATLAB directly beside Rust for each graph family. Rust's best measured setting is 16 CPUs for the weighted path and both worker-firm graphs, and 8 CPUs for the grid and weak-community graphs. MATLAB's best setting is 8 CPUs for dense worker-firm and 16 CPUs for the remaining families. The error bars summarize resampling uncertainty within each implementation; they do not convert this controlled qualification into a population-level hardware claim.

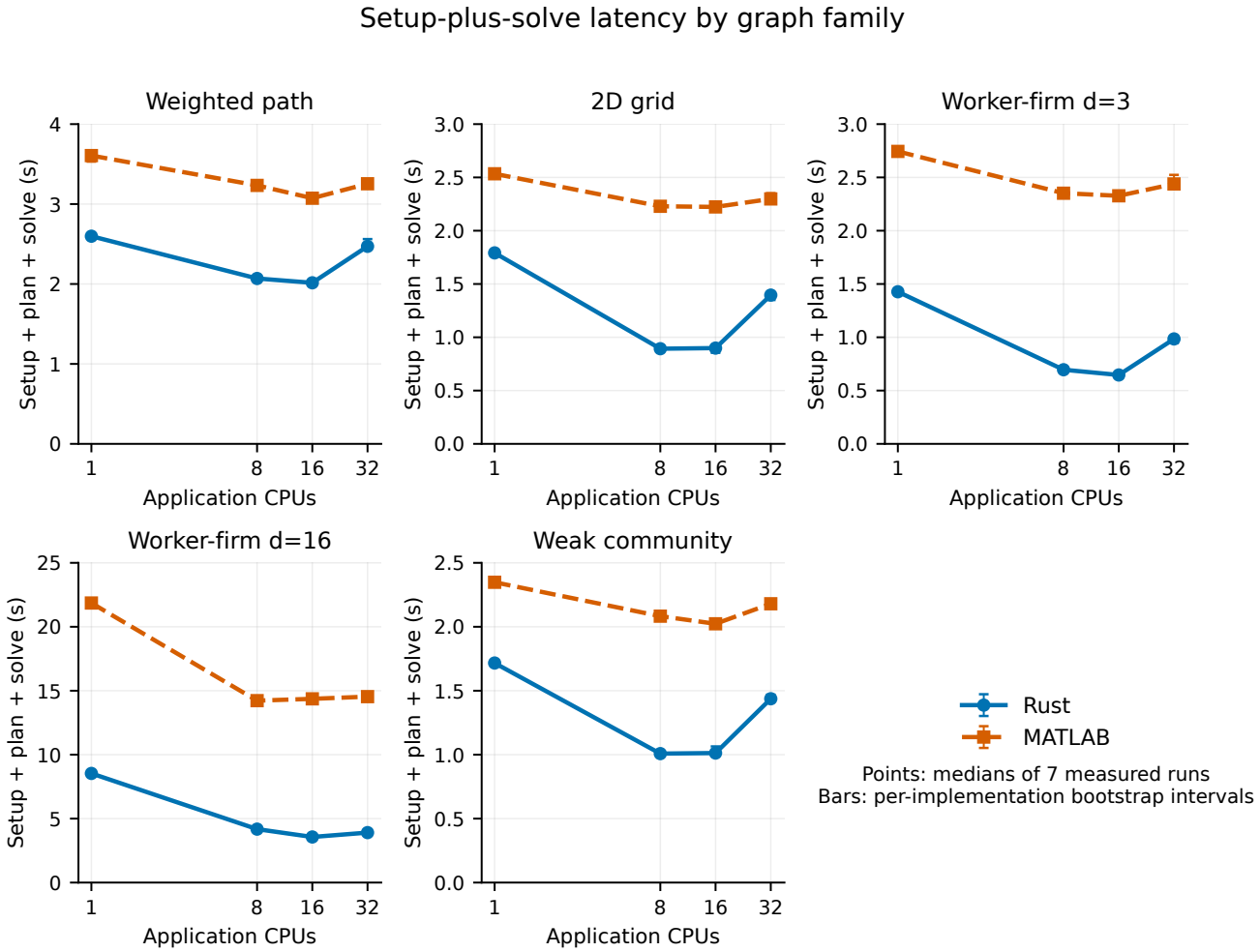


Figure 3: Setup-plus-plan-plus-solve latency by family. Rust and MATLAB share every panel and axis. Points are medians of seven measurements; bars are the reducer's per-implementation bootstrap intervals.

Figure 4 gives the direct ratio view. At 16 CPUs, Rust total time ranges from 0.248 times MATLAB on dense worker-firm to 0.656 times MATLAB on the path. The 16-CPU family results in Table 2 show the absolute times and the different native iteration counts together.

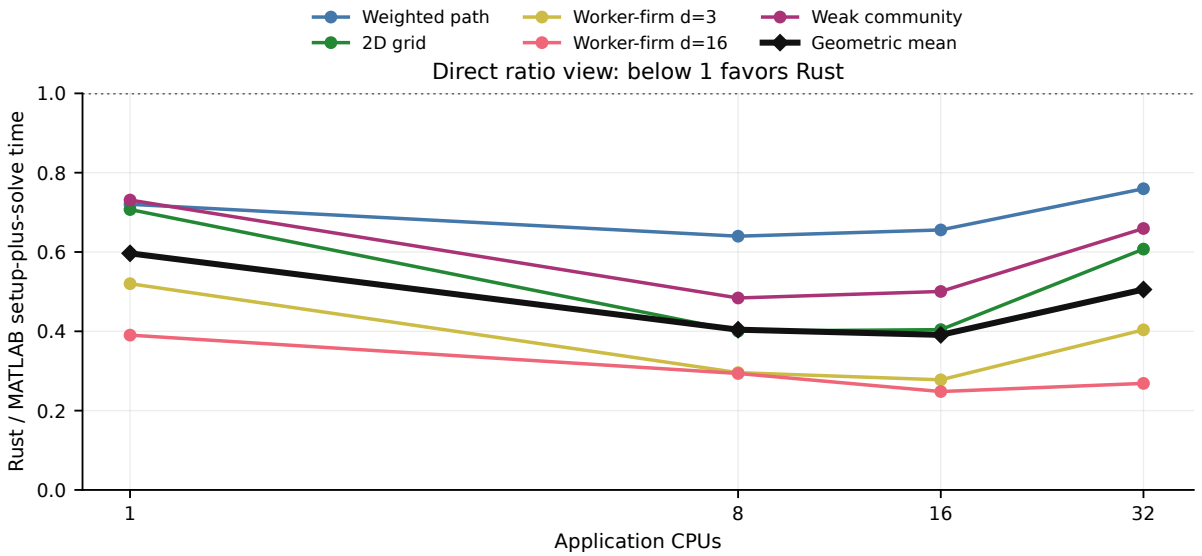


Figure 4: Rust/MATLAB setup-plus-solve ratio for every family and CPU count. Values below one favor Rust.

Table 2: One-million-vertex results at 16 application CPUs.

Family	Edges	Rust (s)	MATLAB (s)	R/M	Rust it.	MATLAB it.
Weighted path	999,999	2.015	3.073	0.656	53	62
2D grid	1,998,000	0.899	2.223	0.404	26	29
Worker-firm d=3	1,499,996	0.646	2.328	0.278	20	23
Worker-firm d=16	7,999,978	3.565	14.366	0.248	9	11
Weak community	1,992,015	1.013	2.023	0.500	26	29

Times are medians. “R/M” is Rust total divided by MATLAB total. Iterations follow each package’s native stopping rule.

6 CPU scaling interpretation

The aggregate Rust total falls from 2.498 seconds at one CPU to 1.334 seconds at 16 CPUs, then rises to 1.803 seconds at 32 CPUs. Every Rust family regresses from 16 to 32 CPUs: the increase ranges from about 10 percent on dense worker-firm to 55 percent on the grid. The likely mechanism is the combined cost of scheduling, synchronization, fixed-order reductions, and memory traffic once additional workers no longer receive enough useful work. This is an inference from the uniform 16-to-32 pattern, not a direct causal decomposition.

For single-right-hand-side latency on Gold 6242, 16 CPUs are therefore the measured aggregate default, with 8 CPUs worth testing on sparse spatial or community graphs. Thirty-two CPUs remain a supported configuration but are not the preferred setting for this workload.

7 Memory comparison

Figure 5 shows Rust and MATLAB together in absolute units and presents the family-level ratios. At 16 CPUs, Rust's geometric-mean peak RSS is 0.227 times MATLAB's. Family ratios range from 0.121 on the path to 0.533 on dense worker-firm. The dense case still uses 3.52 GiB in Rust, so the ratio should not be mistaken for a claim that large dense inputs are memory-free.

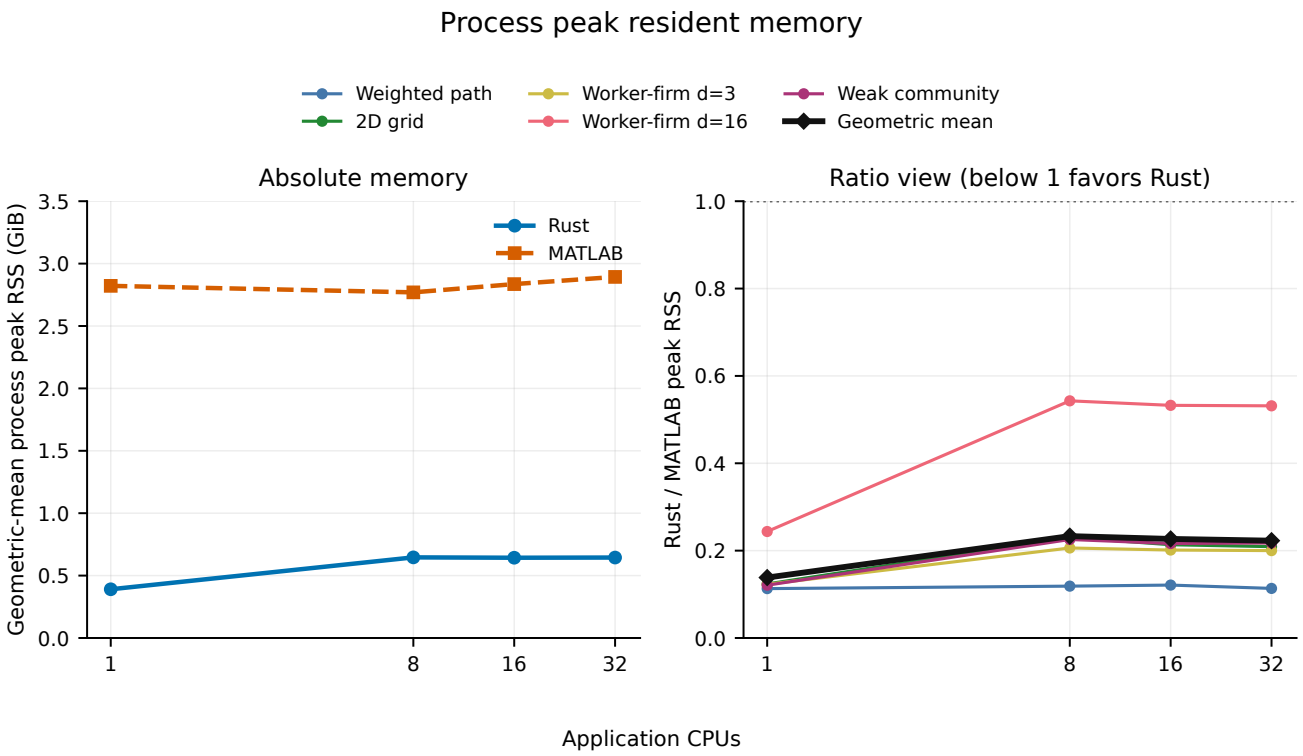


Figure 5: Process peak RSS. Left: geometric-mean absolute memory with Rust and MATLAB on the same axis. Right: family-level Rust/MATLAB ratios and their geometric mean.

8 Numerical validity and iteration counts

All 40 configurations converged and passed the scheduler, identity, timing, and numerical validators. Rust's maximum backward error is 9.24e-09. Its maximum independently recomputed relative residual is 6.76e-08, while MATLAB's maximum native relative residual is 9.73e-09. The larger independent Rust residual is compatible with the package's retained stopping and certification conventions; the report does not claim identical residual definitions.

Rust uses fewer iterations than MATLAB for every family at 16 CPUs (Figure 6), ranging from 9 versus 11 on dense worker-firm to 53 versus 62 on the path. Runtime differences therefore combine implementation speed with the packages' certified native iteration paths. They should not be interpreted as fixed-work microbenchmarks.

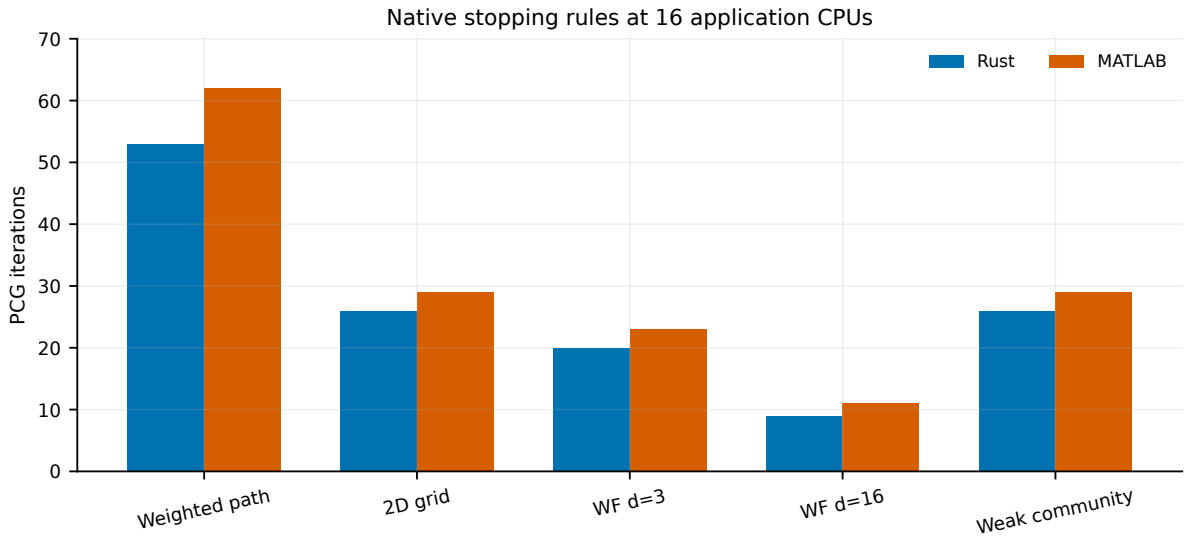


Figure 6: PCG iterations at 16 CPUs under each package's native stopping rule. Iteration counts are invariant across the tested CPU grid within each implementation and family.

MATLAB reports official hierarchy flag 3 for the four dense worker-firm configurations. All four outputs converge and pass the numerical validator. The warnings remain explicit and are not silently discarded or converted into retries.

9 Provenance and acceptance

The accepted array contains five tasks and 40 implementation/CPU configurations. SGE reports `failed=0` and `exit_status=0` for every task; task wall times range from 1,101 to 2,822 seconds and recorded `maxvmem` ranges from 9.05 to 13.47 GiB. Each task also has an application-success receipt, a validation receipt, input hashes, topology, and process-memory records.

Identity	Accepted value
Protocol	`cmg-scc2-v1`
Run	`20260828T021628Z-6fe9be77084a-b2v1-rust-matlab-current`
Rust source	`6fe9be77084a60cca330760361dd4c7addc77ccf`
Source archive SHA-256	`f26148fc0412cd6e4971a29578c81ca284e6cf8bb47f4488406314df4778cbb1`
Rust benchmark binary SHA-256	`759dd3493dab31bc52de48376022c4e92c45eec26ced9b82d134aa25e46f8de5`
Official upstream source	`19752fc102f8cae8e34f66457bfaccb1aaa60375`
Compact accepted record	`.ci/performance/scc-rust-matlab-current.json`
Detailed retained reduction	`benchmarks/report/data/current_results.csv`

The generator requires the complete 5-family by 2-implementation by 4-CPU grid and verifies every row's identity. It recomputes the geometric means and checks them against the compact accepted JSON. PDF compilation stops if a check differs beyond numerical roundoff.